

Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

BAKALÁRSKA PRÁCA

Paralelné programovanie v jazyku Go



2026

Vedúci práce:
Mgr. Jan Laštovička, Ph.D.

Richard Bebjak

Študijný program: Informatika,
Špecializácia: Programovanie a vývoj
software

Bibliografické údaje

Autor:	Richard Bebjak
Názov práce:	Paralelné programovanie v jazyku Go
Typ práce:	bakalárska práca
Pracovisko:	Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci
Rok obhajoby:	2026
Študijný program:	Informatika, Špecializácia: Programovanie a vývoj software
Vedúci práce:	Mgr. Jan Laštovička, Ph.D.
Počet strán:	36
Prílohy:	elektronická data v úložišti katedry informatiky
Jazyk práce:	slovenský

Bibliographic info

Author:	Richard Bebjak
Title:	Parallel programing in language Go
Thesis type:	bachelor thesis
Department:	Department of Computer Science, Faculty of Science, Palacký University Olomouc
Year of defense:	2026
Study program:	Computer Science, Specialization: Programming and Software Development
Supervisor:	Mgr. Jan Laštovička, Ph.D.
Page count:	36
Supplements:	electronic data in the storage of department of computer science
Thesis language:	Slovak

Anotácia

Bakalárska práca sa zaoberá paralelným programovaním v programovacom jazyku Go. Teoretická časť práce predstavuje základné princípy súbežného programovania s dôrazom na jeho využitie pri paralelnom vykonávaní úloh, architektúru jazyka Go a vstavané mechanizmy podporujúce súbežné programovanie. Súčasťou práce je návrh algoritmov využívajúcich tieto mechanizmy a analýza ich správnosti prostredníctvom dôkazov. Praktická časť je zameraná na návrh a implementáciu prototypu chat servera, na ktorom sú demonštrované možnosti súbežného programovania v jazyku Go a využitie navrhnutých algoritmov v praxi.

Synopsis

This bachelor's thesis focuses on parallel programming in the Go programming language. The theoretical part introduces the fundamental principles of concurrent programming, with an emphasis on its application to parallel task execution, the architecture of the Go programming language, and its built-in mechanisms supporting concurrent programming. The thesis also presents the design of algorithms utilizing these mechanisms together with proofs of their correctness. The practical part is devoted to the design and implementation of a prototype chat server that demonstrates the use of Go's concurrency mechanisms and the proposed algorithms in a real-world application.

Kľúčové slová: Go, goroutine, komunikačný kanál, súbežnosť, paralelné programovanie

Keywords: Go, goroutine, channel, concurrency, parallel programming

Děkuji, děkuji, děkuji.

Odevzdaním tohto textu jeho autor čestne vyhlasuje, že celú prácu vrátane príloh vypracoval samostatne a za použitia iba zdrojov spomínaných v texte práce a uvedených v zozname literatúry.

Obsah

1	Úvod	9
2	Communicating Sequential Processes	10
2.1	Komunikácia v CSP	11
2.2	Monitor v CSP	13
3	Jazyk Go	15
3.1	Goroutine	16
3.2	Komunikačné kanály	17
3.2.1	Implementácia	18
3.3	Select	20
3.3.1	Implementácia	21
3.4	Synchronizačné primitíva	22
3.4.1	WaitGroup	22
3.4.2	Mutex a RWMutex	23
4	Chat server	26
4.1	Klientske spojenie	26
4.2	Algoritmus posielania správ	27
4.3	Algoritmus získania správ	29
4.4	Algoritmus príkazu select	30
5	Ďalšie možnosti použitia	32
6	Záver	33
7	Conclusion	34
A	Obsah elektronických dát	35
	Literatúra	36

Zoznam tabuliek

1	Základné prvky syntaxe CSP [1]	12
2	Dátové typy v programovacom jazyku Go	15
3	Štruktúra komunikačného kanála [6]	19

Zoznam obrázkov

1	Pseudokód posielania správ	28
2	Synchronizácia klientov pomocou komunikačného kanála	28
3	Ochrana kritickej sekcie pomocou mutexu	30

Zoznam zdrojových kódov

1	Použitie komunikačného kanála	18
2	Príklad použitia príkazu select	21
3	Použitie WaitGroup	23
4	Použitie RWMutex	24
5	Klientské spojenie	26
6	Prijatie správy	27
7	Poslanie správy	27
8	Získanie správ	29
9	Implementácia príkazu select	30

1 Úvod

Paralelné programovanie predstavuje dôležitú súčasť moderného vývoja softvéru. S rastúcou zložitou softvérových systémov a rozšírením viacjadrových procesorov rastie požiadavka na efektívne využitie výpočtových prostriedkov prostredníctvom paralelného spracovania úloh. Preto pribúdajú riešenia a mechanizmy umožňujúce spracovanie viacerých úloh súčasne. Tradičný prístup založený na zdieľanej pamäti však prináša komplikácie, ako sú chyby súbehu alebo uviaznutie. Z tohto dôvodu vznikajú prístupy založené na synchronizácii procesov prostredníctvom komunikácie a výmeny správ medzi procesmi. Jedným z významných konceptov v oblasti súbežného programovania je model Communicating Sequential Processes (CSP) [1]. Programovací jazyk Go implementuje princípy modelu CSP prostredníctvom goroutín a komunikačných kanálov. Cieľom bakalárskej práce je predstaviť princípy súbežného programovania v jazyku Go, vysvetliť ich teoretické základy vychádzajúce z modelu CSP a demonštrovať ich využitie pri návrhu a implementácii prototypu chat servera. Práca sa venuje teoretickému opisu mechanizmov súbežného programovania, návrhu paralelných algoritmov pre server a analýze ich správnosti prostredníctvom dôkazov.

2 Communicating Sequential Processes

Nasledujúca kapitola vychádza z publikácií [1, 2], ktoré predstavujú základný zdroj informácií o modeli CSP. *Communicating Sequential Processes* (CSP) je formálny model súbežného programovania, ktorý predstavil Charles Antony Richard Hoare v roku 1978. Tento prístup definuje spôsob, akým medzi sebou súbežne vykonávané procesy komunikujú a koordinujú svoje vykonávanie. Na rozdiel od tradičných prístupov založených na zdieľanej pamäti, CSP je založené na komunikácii medzi procesmi, ktorá predstavuje hlavný mechanizmus ich koordinácie a synchronizácie. Koordinácia určuje spôsob spolupráce procesov pri riešení spoločnej úlohy, zatiaľ čo synchronizácia zabezpečuje správne časové usporiadanie ich vykonávania. Procesy si neposkytujú priamy prístup k pamäti, čím sa znižuje riziko vzniku chýb súbehu. Základnou myšlienkou CSP je, že program sa skladá z viacerých sekvenčných procesov vykonávaných súbežne, ktoré medzi sebou komunikujú prostredníctvom vstupno-výstupných operácií.

Model CSP je založený na troch základných princípoch, ktoré umožňujú vytvárať bezpečné súbežné programy:

- strážené príkazy,
- paralelná kompozícia procesov,
- komunikácia pomocou vstupu a výstupu.

Koncept strážených príkazov umožňuje riadiť vetvenie a nedeterministické správanie. Vetvenie znamená výber jednej z viacerých možných vetiev programu podľa splnenia podmienky. Nedeterministické správanie nastáva v situácii, keď je súčasne splnených viac podmienok a model nepredpisuje, ktorá z možných vetiev bude vykonaná. Dôležité je iba to, že bude zvolená jedna z povolených možností. Ide o príkazy, ktorých vykonanie je podmienené splnením logickej podmienky. Ak podmienka nie je splnená, proces čaká, kým je podmienka splnená.

Paralelná kompozícia umožňuje vytvoriť program pozostávajúci z viacerých procesov, ktoré sa vykonávajú súčasne, nezávisle od seba a končia až po ukončení všetkých procesov.

Komunikácia pomocou vstupu a výstupu znamená, že procesy spolu komunikujú výhradne pomocou vstupno-výstupných operácií. Výstupná operácia predstavuje odoslanie hodnoty procesom, zatiaľ čo vstupná operácia predstavuje prijatie hodnoty iným procesom. Komunikácia sa uskutoční iba vtedy, keď sú odosielateľ aj prijímateľ pripravení vykonať príslušné operácie. Každá komunikácia predstavuje synchronizačný bod medzi dvoma procesmi, pretože prenos údajov prebieha iba vtedy, keď sú odosielateľ aj prijímateľ pripravení komunikovať. Procesy tak nezdieľajú pamäť a ich komunikácia prebieha synchronizovane.

V CSP možno chápať každý proces ako samostatnú výpočtovú jednotku s vlastným stavom, ktorá vykonáva príkazy v definovanom poradí. Na opis správania procesu sa používa pojem *udalosť*. Udalosť predstavuje pozorovateľnú akciu procesu. Nie je dôležitá implementácia, ale navonok pozorovateľné správanie.

Udalosť predstavuje okamih pozorovateľný ostatnými procesmi. Postupnosť udalostí sa označuje ako *stopa*. Stopa predstavuje chronologický záznam všetkých udalostí, ktoré proces od svojho vzniku vykonal. Správanie procesu je možné opísať pomocou postupnosti udalostí, ktoré proces počas svojho vykonávania vyvolá, bez potreby poznať jeho implementáciu. Tento spôsob opisu je jeden zo základných princípov formálneho modelovania v CSP. Procesy nie sú izolované. Ich spolupráca je kľúčová na dosiahnutie požadovaného výsledku a je realizovaná pomocou synchronizovanej komunikácie. Všetky informácie získava alebo odosiela prostredníctvom výmeny správ. Tento prístup ale môže viesť k situácii, kedy procesy čakajú bez možnosti pokračovať, teda uviaznutiu pre jeho správanie, kedy odosielateľ čaká na príjemcu a naopak.

2.1 Komunikácia v CSP

Hlavnou vlastnosťou CSP je prenos dát medzi procesmi prostredníctvom zasielania správ. Hodnota odoslaná jedným procesom je priamo priradená premennej v druhom procese. Procesy musia byť pripravené na komunikáciu v rovnakom okamihu. Proces nemôže pokračovať, kým druhý proces nie je pripravený na komunikáciu. Prenos hodnoty je atomický a synchronizovaný, teda nemôže prísť k čiastočnému prenosu alebo nekonzistentnému stavu. Komunikácia medzi procesmi súčasne zabezpečuje prenos dát aj synchronizáciu procesov. Tento spôsob komunikácie zjednodušuje uvažovanie o správnosti programu, pretože eliminuje potrebu zamykania a konflikty pri prístupe k pamäti.

Na opis správania procesov CSP používa vlastnú syntax.

Tabuľka 1: Základné prvky syntaxe CSP [1]

Syntax	Význam
$a := b$	Priradenie hodnoty premennej.
$P?x$	Prijatie hodnoty od procesu P a uloženie do premennej x .
$P!x$	Odoslanie hodnoty x procesu P .
$SKIP$	Prázdny príkaz, ktorý nevykonáva žiadnu operáciu a okamžite končí.
$A \rightarrow B$	Po vykonaní príkazu A pokračuje proces vykonávaním príkazu B . Tento zápis predstavuje prefixový operátor v CSP.
$G \rightarrow S$	Strážžený príkaz (guard). Ak je splnená podmienka G , vykoná sa príkaz S . V opačnom prípade proces čaká, kým sa podmienka nestane pravdivou.
$[]$	Alternatívny príkaz. Vyberie jednu zo strážžených alternatív, ktorých podmienka je splnená. Ak je splnených viac podmienok, výber je nedeterministický.
$*[]$	Repetitívny príkaz. Opakovane vykonáva alternatívny príkaz, pokiaľ je splnená aspoň jedna podmienka.
$(i : 1..n)$	Rozsah indexov používaný pri komunikácii s ľubovoľným procesom zo skupiny procesov.
$P Q$	Paralelné vykonávanie procesov P a Q .
μX	Rekurzívna definícia procesu umožňuje definovať proces odkazujúci na svoju vlastnú definíciu, čím možno modelovať opakujúce sa alebo nekonečné správanie.
$\dots$	Ľubovoľná činnosť vykonaná procesom po úspešnej komunikácii, ktorej implementácia nie je z pohľadu modelu CSP podstatná.

Nasledujúci text sa zameriava iba na tie prvky syntaxe CSP, ktoré sú využité v ďalších častiach práce. Ostatné konštrukcie sú uvedené v tabuľke pre úplnosť. Priradenie hodnoty premennej má rovnaký význam ako v bežných imperatívnych programovacích jazykoch. Vstup a výstup sú synchronizované tak, že proces posielať výstupnú operáciu musí čakať, kým iný proces nevykoná zodpovedajúcu vstupnú operáciu. Napríklad, ak proces P odošle hodnotu operáciou $P!x$, druhý proces musí byť pripravený prijať túto hodnotu operáciou $P?y$. Až potom dôjde k samotnému prenosu údajov. Obidve operácie tvoria jednu spoločnú komunikačnú udalosť.

Štruktúrované príkazy predstavujú zloženejšie riadiace konštrukcie vytvorené z jednoduchých príkazov. Štruktúrované príkazy sú zložené z alternatívnych príkazov alebo repetitívnych príkazov.

Alternatívny príkaz slúži na výber jednej z viacerých možných alternatív podľa splnenia strážených podmienok. Nasledujúci príklad demonštruje použitie alternatívneho príkazu v CSP.

$$\begin{array}{l} [\\ x > 0 \rightarrow P!x \\ y > 0 \rightarrow Q!y \\] \end{array}$$

Ak je splnená iba podmienka $x > 0$, vykoná sa prvá alternatíva. Ak je splnená iba podmienka $y > 0$, vykoná sa druhá alternatíva. Ak sú splnené obe podmienky, model CSP nedeterminizmom vyberie jednu z nich. Ak nie je splnená ani jedna podmienka, proces čaká, kým sa niektorá z podmienok nestane pravdivou. Alternatívny príkaz slúži na modelovanie situácií, kde správanie programu závisí od aktuálnych podmienok alebo interakcie s inými procesmi.

Repetitívny príkaz predstavuje cyklus alternatívnych príkazov. Princíp fungovania repetitívneho príkazu je znázornený v nasledujúcom príklade.

$$\begin{array}{l} * [\\ x > 0 \rightarrow x := x - 1 \\ y > 0 \rightarrow y := y - 1 \\] \end{array}$$

Proces opakovane vyhodnocuje podmienky $x > 0$ a $y > 0$, kým je možné vykonať aspoň jednu z alternatív. Ak už nie je splnená žiadna podmienka alebo je proces zastavený, vykonávanie sa ukončí. Repetitívny príkaz sa používa na modelovanie situácií, kde program opakovane reaguje na udalosti.

2.2 Monitor v CSP

Pri riešení zložitejších úloh často komunikuje viacero procesov s jedným spoločným procesom, ktorý koordinuje ich činnosť. Takýto proces možno chápať ako monitor. Monitor ako synchronizačný objekt prijímajúci požiadavky od ostatných procesov, zabezpečujúci ich vzájomnú synchronizáciu a rozhodujúci o poradí ich spracovania. V modeli CSP monitor nepredstavuje zdieľanú pamäť, ale samostatný proces komunikujúci prostredníctvom výmeny správ. Procesy musia byť odlišne pomenované napr. „Výrobca“ a „Spotrebiteľ“, alebo mať odlišný index napríklad „X(0)“. Každá komunikácia musí mať jednoznačne určený svoj zdroj alebo cieľ. Keď je monitor pripravený komunikovať s ktorýmkoľvek používateľským procesom, zväčša ktorý príde prvý, používa strážený príkaz s rozsahom. Predpokladajme, že existuje sto procesov označených ako X(1) až X(100), ktoré komunikujú s jedným monitorom. Ich vnútorná implementácia nie je pre tento príklad podstatná. Dôležitá je iba udalosť komunikácie medzi procesom a monitorom, prostredníctvom ktorej proces odovzdáva požiadavku a

prijíma výsledok. Monitor nevie dopredu, ktorý proces odošle požiadavku ako prvý. Preto musí byť pripravený prijať správu od ktoréhokoľvek z nich.

$$*[(i : 1..100); X(i)?(value\ parameters) \rightarrow \dots; X(i)!(results);]$$

Premenná i identifikuje proces, ktorý komunikáciu inicioval. Monitor po prijatí správy vykoná potrebnú činnosť, ktorej implementácia nie je podstatná a následne odošle výsledok späť správne procesu.

Ak monitor nie je pripravený na komunikáciu od konkrétneho procesu napríklad $X(i)$, môže byť vstupný príkaz doplnený o booleovskú podmienku, takzvanú stráž. Napríklad možno zabrániť tomu, aby monitor dvakrát po sebe obslúžil požiadavku od toho istého procesu. To možno dosiahnuť podmienkou $i \neq j$, kde premenná j uchováva identifikátor naposledy obslúženého procesu. Predstavme si, že monitor obsluhuje požiadavky klientov. Ak by jeden klient posielal požiadavky nepretržite, ostatné procesy by mohli zostať dlhý čas neobslúžené. Preto možno pomocou stráženej podmienky dočasne zablokovať opätovné prijatie požiadavky od toho istého procesu, kým nebude spracovaná požiadavka od iného procesu.

$$*[(i : 1..100); j \neq i \rightarrow X(i)?(parameters) \rightarrow \dots; j := i;]$$

V tomto prípade monitor neprijme ďalšiu požiadavku od procesu $X(i)$, kým nebude spracovaná požiadavka iného procesu. Rovnakým spôsobom možno pomocou strážených podmienok odložiť spracovanie požiadaviek, ktoré by v danom okamihu porušili pravidlá plánovania alebo iné obmedzenia systému.

Ako príklad možno uviesť problém výrobcu a spotrebiteľa. Predpokladajme, že existuje viacero procesov „Výrobca“, ktoré vytvárajú údaje, a viacero procesov „Spotrebiteľ“, ktoré ich spracúvajú. Medzi nimi sa nachádza monitor. Výrobci odosielať vytvorené údaje monitoru, zatiaľ čo spotrebiteľia od monitora požadujú ďalšiu dostupnú položku. Monitor zabezpečuje, aby sa údaje odovzdávali korektne aj v prípade, že viac výrobcov alebo spotrebiteľov komunikuje súčasne.

$$\begin{aligned} Výrobca(i) &= [Monitor!item \rightarrow \dots] \\ Spotrebiteľ(j) &= [Monitor?item \rightarrow \dots] \\ Monitor &= *[(i : 1..m); (j : 1..n); \\ &\quad Výrobca(i)?item \rightarrow Spotrebiteľ(j)!item;] \end{aligned}$$

V uvedenom príklade môže existovať ľubovoľný počet výrobcov a spotrebiteľov. Výrobca odošle správu prostredníctvom synchronizovanej komunikácie Monitoru a po úspešnom dokončení pokračuje ďalej vo vykonávaní ďalších operácií. Analogicky Spotrebiteľ' pokračuje vo vykonávaní ďalších operácií až po úspešnom prijatí položky od Monitora. Indexy i identifikujú konkrétny proces Výrobca a j identifikujú konkrétny proces Spotrebiteľ'. Monitor opakovane prijíma údaje od ľubovoľného výrobcu z množiny procesov označenej rozsahom $i : 1..m$, ktorý je pripravený na komunikáciu a následne ich poskytuje ľubovoľnému spotrebiteľovi z množiny $j : 1..n$, pričom zabezpečuje správnu synchronizáciu komunikácie medzi všetkými procesmi.

3 Jazyk Go

Programovací jazyk Go bol vyvinutý spoločnosťou Google s cieľom vytvoriť jednoduchý, efektívny a výkonný jazyk. Jazyk je vhodný na vývoj moderných aplikácií. Pri návrhu bol kladený dôraz na rýchlosť kompilácie, jednoduchú syntax, bezpečnosť práce s pamäťou a natívnu podporu súbežného programovania. Go patrí medzi kompilované, staticky typované programovacie jazyky a poskytuje automatickú správu pamäte pomocou garbage collectoru. [3] Go obsahuje vstavané mechanizmy na paralelné a súbežné programovanie, ktorým sa venuje táto práca.

Go je staticky typovaný jazyk, čo znamená, že typ každej premennej je známy už počas kompilácie. Vďaka tomu je možné odhaliť chyby s tým spojené ešte pred spustením programu. Typ premennej môže byť uvedený explicitne:

```
var variable_name type = value
```

alebo pomocou skrátenej deklarácie, ktorú vie kompilátor odvodiť:

```
variable_name := value
```

Skrátená deklarácia patrí medzi najčastejšie používané zápisy v Go a znižuje množstvo potrebného kódu. [3]

Go podporuje dátové typy:

Tabuľka 2: Dátové typy v programovacom jazyku Go

celočísenné typy	int, int64, uint
reálne čísla	float32, float64
logický typ	bool
reťazce	string
polia	array
rezy	slice
mapy	map
štruktúry	struct

Funkcie sa deklarujú pomocou kľúčového slova *func*. Návratový typ sa zapisuje za zoznam argumentov. [3]

```
func name(arguments_name argument_type) return_type {  
    return value  
}
```

Významnou vlastnosťou Go je možnosť viacerých návratových hodnôt z funkcie. Často sa využíva pri návrate výsledku spolu s informáciou o chybe:

```
first_value, second_value := ReadFile(path)
```

Takýto spôsob spracovania chýb znižuje potrebu používania výnimiek a predstavuje štandardný mechanizmus ošetrovania chýb v Go. [3]

Jedným z hlavných cieľov Go je jednoduchosť. Jazyk preto neobsahuje preťažovanie funkcií, preťažovanie operátorov, implicitné pretypovanie dátových typov ani aritmetiku ukazovateľov. Natívne podporuje mechanizmy súbežného a paralelného programovania. Poskytuje primitíva ako goroutiny, komunikačné kanály, synchronizačné primitíva v balíku `sync` a príkaz `select`. [4]

3.1 Goroutine

Goroutine (*goroutine*) je ľahká vykonávacia jednotka, ktorá umožňuje súbežné vykonávanie funkcií v jazyku Go. Na rozdiel od vlákien operačného systému nie je priamo spravovaná operačným systémom, ale runtime prostredím jazyka Go, ktoré zabezpečuje ich vytváranie, plánovanie a vykonávanie. Obsahujú však body, v ktorých môžu byť pozastavené a znovu spustené. Na rozdiel od vlákien je prepínanie medzi gorutinami rýchlejšie približne o 1,242 μ s.[5] Goroutiny sa dajú spustiť pomocou kľúčového slova *go*.

```
func main{
    go SayHello()
}

func SayHello(){
    fmt.Println("Hello")
}
```

Gorutiny možno implementovať ako anonymné funkcie.

```
go func(){
    fmt.Println("Hello")
}
```

Pri použití anonymnej funkcie je potrebné funkciu bezprostredne zavolať.

Vieme goroutinu priradiť do premennej.

```
sayHello := func(){
    fmt.Println("Hello")
}

go sayHello()
```

Goroutiny sú špecifické pre jazyk Go, pretože sú integrované do runtime prostredia Go. Goroutiny si sami neurčujú body pozastavenia. Runtime Go sleduje ich správanie a automaticky ich pozastaví a znovu spustí, keď sú opätovne pripravené pokračovať. Súbežné vykonávanie gorutín zabezpečuje plánovač runtime prostredia Go. [3]

V jazyku Go je možné pomocou kľúčového slova *defer* zaregistrovať volanie funkcie, ktoré sa vykoná tesne pred ukončením aktuálnej gorutiny. Odložené volanie sa vykoná bez ohľadu na spôsob ukončenia funkcie, či už dôjde k jej prirodzenému dokončeniu alebo k návratu prostredníctvom príkazu *return*.

```

go func(){
    defer close(channel)
}()

```

Go používa plánovač $M:N$ a model súbežnosti *fork-join*.^[5] Teda na M vláknach sa vykonáva N goroutín. Goroutiny sú mapované na tieto vlákna a go runtime sa stará o ich rozdelenie a efektívne vykonávanie. Stará sa o ich spúšťanie, ak je nejaká goroutina blokováaná, runtime môže spustiť inú. Podľa modelu fork-join, *Fork* znamená, že program môže vytvoriť novú goroutinu, ktorá sa vykonáva súbežne s rodičovským kódom. Naopak, *Join* označuje, že sa tieto vetvy spoja v mieste nazývanom join point. Samotné vytvorenie goroutiny nezabezpečuje čakanie na jej dokončenie.

3.2 Komunikačné kanály

Komunikačné kanály predstavujú jeden zo základných mechanizmov synchronizácie a komunikácie medzi goroutinami. Sú zabudované referenčným typom, ktorý umožňuje bezpečný prenos dát. Ich návrh vychádza z Communicating Sequential Processes, ktorý zdôrazňuje komunikáciu medzi procesmi miesto zdieľania pamäte.^[5]

Komunikačné kanály sa deklarujú pomocou kľúčového slova *chan* a vždy sa deklarujú aj s určitým typom. To znamená, že kanál môže prenášať hodnoty iba tohto typu. Komunikačné kanály patria k referenčným typom, preto ich deklarácia nevytvára potrebnú pamäťovú štruktúru. Pamäť sa alokuje pomocou zabudovanej funkcie *make*.

```
make(chan type, buffer_size)
```

Takto vytvorený komunikačný kanál umožňuje zapisovanie a čítanie hodnôt.

Každý komunikačný kanál môže byť bufferovaný alebo nebufferovaný. Nebufferovaný komunikačný kanál nemá žiadnu kapacitu a každá operácia zápisu musí byť okamžite spravovaná operáciou čítania.^[6]

Bufferovaný komunikačný kanál sa definuje pomocou voliteľného druhého argumentu funkcie *make*. Bufferovaný komunikačný kanál obsahuje buffer s definovanou kapacitou, ktorá umožní uložiť určitý počet hodnôt do buffera bez okamžitého čítania. Bufferovaný komunikačný kanál zablokuje zápis až v momente keď je buffer plný.

Jazyk Go umožňuje definovanie funkcií, kde ako parameter definujeme komunikačný kanál. Pri týchto funkciách je umožnené definovanie jednosmerných kanálov, ktoré podporujú iba zápis alebo čítanie. Tieto komunikačné kanály sa používajú ako parametre funkcií, aby určili akým spôsobom sa komunikačný kanál v danej časti programu používa. Tento mechanizmus zvyšuje bezpečnosť programu, pretože kompilátor dokáže zachytiť nesprávne použitie.

Zápis do komunikačného kanála umožňuje operátor $<-$, ktorý sa umiestňuje na pravú stranu názvu komunikačného kanála, pre čítanie sa operátor umiestňuje na ľavú stranu názvu komunikačného kanála.

```
channel <- data
variable = <- channel
```

Dôležitou vlastnosťou komunikačného kanála je jeho blokujúce správanie. Ak goroutina zapisuje do komunikačného kanála, ktorý je plný, je jej proces blokovaný kým iná goroutina z komunikačného kanála hodnotu neprečíta. Podobne čítanie z prázdneho komunikačného kanála je blokované kým nie je do komunikačného kanála zapísaná nová hodnota.

Komunikačný kanál je možné zavrieť pomocou funkcie *close*. Pri čítaní z kanála je možné získať druhú návratovú hodnotu typu *bool*, ktorá indikuje jeho otvorenie. Uzavretie teda signalizuje prijímateľovi, že do komunikačného kanála nebude viac zapisované. Ak odosielať pošle dáta do uzavretého komunikačného kanálu, program vráti error *panic: send on closed channel*. Po uzavretí komunikačného kanálu, prijímateľ nečaká na hodnotu ale obdrží prázdnu hodnotu a hodnotu *False* z druhej návratovej hodnoty. Nasledujúci príklad demonštruje použitie.

```
1      var channel = make(chan int , 1)
2
3      go func(){
4          channel <- 42
5      }()
6
7      result := <- channel
8      close(channel)
```

Ukážkový kód 1: Použitie komunikačného kanála

Ako je vidieť na ukážkovom kóde 1 na riadku jedna v si vytvoríme komunikačný kanál s referenčným typom *int* a veľkosťou buffera jedna. Na riadku tri spustíme novú goroutinu. Na riadku štyri v goroutine uložíme do komunikačného kanála hodnotu 42. Na riadku sedem hlavné vlákno čaká na dáta z komunikačného kanála. Ak dáta prídu, uloží do premennej *result* dáta z komunikačného kanála, v tomto prípade 42. Na riadku osem uzavrieme komunikačný kanál.

3.2.1 Implementácia

V runtime prostredí je komunikačný kanál implementovaný ako štruktúra *hchan*. Obsahuje počet hodnôt v kanáli, maximálnu kapacitu, ukazovateľ na buffer, fronty čakajúcich goroutín a synchronizačný zámok. Zjednodušene štruktúra obsahuje:

Tabuľka 3: Štruktúra komunikačného kanála [6]

qcount uint	Počet prvkov v bufferi
dataqsiz uint	Veľkosť jednej hodnoty na bufferi
buf unsafe.Pointer	Pointer na buffer
sendx uint	Index pre zápis
recvx uint	Index pre čítanie
recvq waitq	Čakajúci prijímateľa
sendq waitq	Čakajúci odosielateľa
closed uint32	Hodnota určujúca či je komunikačný kanál otvorený alebo zatvorený
lock mutex	Mutex na prístup k bufferu

Komunikačný kanál funguje ako kruhový buffer, kde dva indexy označujúce aktuálnu pozíciu a pozíciu posledného pridaného prvku. Pri dosiahnutí maximálneho indexu, ktorý určuje možný počet prvkov v bufferi, sa pokračuje indexom nula. Ak sú indexy na rovnakej pozícii, značí to prázdny buffer. Ak je index posledného pridaného prvku o jedna menší ako index aktuálnej pozície alebo index posledného pridaného prvku je maximálny a index aktuálnej pozície je nula, značí to plný buffer. Komunikačný kanál je doplnený o mechanizmy blokujúce goroutiny.[6] Implementácia front `sendq` a `recvq` je hlavný prvok na synchronizáciu goroutín, kedy odosielateľ čaká na príjemcu a príjemca čaká na odosielateľa. To zabezpečuje bezpečnú synchronizáciu a absenciu problému súbehu zámkov v aplikačnom kóde. Záмок `mutex` využíva komunikačný kanál pri operáciách, ktoré môžu meniť jeho stav. Teda pri posielaní alebo odosielaní správ sa komunikačný kanál uzamkne aby nenastal nekonzistentný stav komunikačného kanála.

Komunikačný kanál má operácie prijímanie a odosielanie hodnoty. Komunikačný kanál odosiela a prijíma dve hodnoty. Posielanú hodnotu a hodnotu typu `bool`, ktorá značí, či je kanál otvorený. Odosielanie hodnoty je implementované funkciou `chansend`. Pri tejto operácii je komunikačný kanál uzamknutý, preto túto operáciu možno vykonať iba jedna goroutina v určitom čase. Jej správanie závisí od aktuálneho stavu komunikačného kanála. Zjednodušene, rozlišuje tri stavy. Ak existuje čakajúci príjemca, teda je hodnota v `recvq` goroutine, hodnota sa preniesie priamo bez použitia bufferu.

```
if receiver exist:
    copy data directly to receiver
    wake up receiver
```

Najrýchlejší scenár, nakoľko sa neukladá hodnota do bufferu. Ďalší stav, ak neexistuje čakajúci príjemca ale existuje voľné miesto v bufferi. Tak sa hodnota uloží na buffer a zvýši sa počítadlo.

```
if buffer not full:
    store value in buffer
    increment qcount
```

Tretí stav ak neexistuje čakajúci príjemca a neexistuje voľné miesto na bufferi. V takom prípade sa goroutina uloží do fronty `sendq` a čaká na uvoľnenie miesta na bufferi.

Prijímanie hodnoty je implementované funkciou `chanrecv`. Rovnako jej správanie závisí od aktuálneho stavu komunikačného kanála. Zjednodušene rozlišuje tri stavy. Ak existuje čakajúci odosielateľ, teda je hodnota v `sendq`, hodnota sa prenesie priamo bez použitia bufferu.

```
if sender exist:
    receive value directly
    wake sender
```

Ak neexistuje čakajúci odosielateľ ale buffer obsahuje aspoň jednu hodnotu, je táto hodnota načítaná a zníži sa hodnota počítadla.

```
if buffer not empty:
    read value from buffer
    decrement qcount
```

Ak neexistuje čakajúci odosielateľ a buffer neobsahuje žiadnu hodnotu. V takom prípade sa goroutina uloží do fronty `recvq` a čaká na odosielateľa.

Operácia uzavretia komunikačného kanála nastaví `closed` na hodnotu `False`. Zároveň sa odblokujú všetky čakajúce goroutiny v `recvq`. Goroutiny v prvej návratovej hodnote obdržia prázdnu hodnotu, čo nie je vždy chcené a je potrebná reakcia programu na druhú návratovú hodnotu. Pri odosielaní na uzavretý komunikačný kanál, program vráti chybu.

3.3 Select

Príkaz `select` slúži na koordináciu komunikácie medzi goroutinami. Zatiaľ čo komunikačný kanál slúži na synchronizáciu a prenos dát, príkaz `select` umožňuje efektívne kombinovať a riadiť komunikačné kanály na základe aktuálneho stavu programu. Používa sa na spájanie viacerých komunikačných kanálov, synchronizáciu, implementáciu časových limitov alebo reakciu na viacero udalostí súčasne. `Select` sa používa v spojení s komunikačnými kanálmi a má podobnú syntax ako príkaz `switch`. [5] Pozostáva z príkazu *select* a viacerých vetiev *case*, kde každá vetva reprezentuje operáciu nad kanálom. Môže to byť operácia čítania alebo zápisu. Jeho hlavnou vlastnosťou pre synchronizáciu je jeho blokovacie správanie. Proces je blokován kým nie je pripravená na vykonanie aspoň jedna z jeho vetiev. Pripravenosť znamená, že pri zápise do komunikačného kanála, nie je komunikačný kanál zaplnený a pri čítaní z komunikačného kanála, komunikačný kanál obsahuje dáta alebo je uzavretý. Pre neblokujúce správanie príkazu `select` slúži jeho posledná vetva *default*, ktorá sa vykoná v prípade, ak nie je pripravená na vykonanie žiadna z ostatných vetiev. [7] Rozdiel oproti príkazu `switch` je, že jednotlivé vetvy sa nevyhodnocujú postupne ale všetky súčasne. Ak je pripravená niektorá z vetiev, tak sa vykoná. Ak je pripravených viacero vetiev v rovnakom okamihu, runtime Go vyberie jednu z pripravených a vykoná ju. Výber

je rovnomerne rozdelený medzi všetky pripravené vetvy, preto má každá rovnakú pravdepodobnosť vykonania[5]. Príkaz `select` umožňuje implementáciu časových limitov. Pomocou funkcie `time.After` z balíka `time`, ktorá vracia komunikačný kanál aktivovaný po uplynutí zadaného času. [8] Tento prístup umožňuje riešenie situácií, kde nie je žiaduce čakať neobmedzene dlho.

```
1      ch := make(chan int , 1)
2      go func() {
3          for {
4              i := rand.Intn(5)
5              ch <- i
6              time.Sleep(time.Duration(i) * time.Second)
7          }
8      }()
9
10     for {
11         select {
12             case num := <-ch:
13                 fmt.Println(num)
14
15             case <-time.After(3 * time.Second):
16                 return
17         }
18     }
```

Ukážkový kód 2: Príklad použitia príkazu `select`

V príklade 2 sme si na riadku jedna vytvorili komunikačný kanál a na riadkoch dva až osem jednoduchú funkciu, ktorá vyberie náhodné číslo od nula do štyri, pošle ho cez komunikačný kanál a počká taký počet sekúnd. Funkcia je spustená v novej goroutine. V nekonečnom cykle na riadku jedenásť máme príkaz `select`. Prvý `case` na riadku dvanásť je blokovaný kým nie je cez komunikačný kanál poslaná hodnota. Po prijatí hodnoty ju vypíše. Druhý `case` na riadku patnáť je blokovaný pri každej interakcii cyklu tri sekundy, po uplynutí troch sekúnd sa vykoná vetva na riadku šestnásť a proces sa ukončí. S každou interakciou sa vytvorí nový časovač. Na tom môžeme vidieť, že `select` si vyhodnocuje všetky vetvy súčasne. Ak je aspoň jedna pripravená na vykonanie, vykoná ju, inak je blokovaný.

3.3.1 Implementácia

V runtime Go je vetva `case` v príkaze `select` reprezentovaná štruktúrou `scase`, ktorá obsahuje ukazovateľ na komunikačný kanál a nepovinný ukazovateľ na prenášané dáta.[7]

```

type scase struct {
    c *hchan
    elem unsafe.Pointer
}

```

Vetvy sú následne uložené do poľa, kde sa periodicky testuje prvok na náhodnej pozícii. Používa sa na to funkcia *cheaprandn*. Použitím náhodného testovania vetiev sa zabezpečí rovnaká pravdepodobnosť vykonania pre každú vetvu. Pred vykonaním výberu sa ale uzamkne príslušný komunikačný kanál aby sa predišlo uviaznutiu.

Najprv *select* hľadá operáciu, ktorá je pripravená na vykonanie bez blokovania goroutín s príkazom *select*. Teda snaží sa vykonať príslušnú operáciu na komunikačný kanál bez toho, aby sa zapísala do fronty čakateľov, *recvq* pre operáciu prijatia a *sendq* pre operáciu odoslania. Ak nie je pripravená žiadna vetva na vykonanie tak sa goroutina s príkazom *select* zapíše do každého komunikačného kanála do fronty čakateľov. Následne je blokováná až kým ju neprebudí jeden z komunikačných kanálov. V takom prípade sa odstráni z fronty čakateľov u ostatných kanálov a pokračuje operáciou výberu. Ak je *select* použitý s vetvou *default* a žiadna operácia nie je pripravená, vykoná sa vetva *default* bez blokovania.

3.4 Synchronizačné primitíva

V podkapitole čerpáme z [9]. Jazyk Go poskytuje *sync* balík, ktorý obsahuje synchronizačné primitíva. Tieto synchronizačné primitíva sú používané pri nízkoúrovňovom prístupe do pamäte. Rozdiel medzi ostatnými jazykmi a jazykom Go je, že Go poskytuje novú sadu synchronizačných primitív pre súbežnosť nad sadou primitív pre synchronizáciu prístupu k pamäti.

3.4.1 WaitGroup

WaitGroup je nástroj z balíka *sync*, ktorého účelom je umožniť programu počkať na dokončenie skupiny goroutín pred tým, než bude pokračovať. Mechanizmus sa využíva v prípade, že sa vykonáva viacero nezávislých úloh. *WaitGroup* si môžeme predstaviť ako bezpečný čítač. Tento čítač eviduje počet goroutín, ktoré ešte nedokončili svoju činnosť. Čítač sa zvyšuje pomocou metódy *Add(n)*, kde „n“ udáva počet goroutín, ktoré sa majú vykonať. Čítač sa znižuje pomocou metódy *Done()*. Program je zablokovaný na metóde *Wait()*, kým hodnota čítača nedosiahne nulovú hodnotu. Nasledujúci príklad demonštruje jeho použitie.

```

1      var wg sync.WaitGroup
2
3      for i := 0; i < 10; i++ {
4          wg.Add(1)
5          go func(i int) {
6              defer wg.Done()
7              fmt.Println("Goroutine", i)
8              time.Sleep(200)
9          }(i)
10     }
11     wg.Wait()
12     fmt.Println("End")

```

Ukážkový kód 3: Použitie WaitGroup

V ukážkovom kóde 3 na prvom riadku si definujeme WaitGroup. Následne v cykle na riadku štyri zvýšime hodnotu WaitGroup o jedna. Na riadku päť pustíme novú goroutinu. V goroutine na riadku šesť zabezpečíme, že po dokončení činnosti, zníži hodnotu WaitGroup o jedna. Následne vykoná výpis a čakanie pre simuláciu paralelného výpočtu. Na riadku jedenásť program čaká, kým hodnota WaitGroup bude nulová. Následne vypíše End.

3.4.2 Mutex a RWMutex

Mutex je nástroj z balíka sync, ktorého účelom je chrániť časť programu so zdieľanými zdrojmi, ku ktorým môže pristupovať iba jedna goroutina. Pred vstupom do časti programu so zdieľanými zdrojmi, musí goroutina získať zámok pomocou metódy *Lock()*. Po ukončení činnosti v danej časti programu, uvoľní zámok pomocou metódy *Unlock*. Počas držania zámku nemôže iná goroutina vstúpiť do tejto časti programu. RWMutex rozširuje Mutex o zámok, ktorý rozlišuje operácie čítania a zápisu. Metódy na získanie zámku *RLock()* / *Lock()*, metódy na uvoľnenie zámku *RUnlock()* pre čítanie a *Unlock()* pre zápis. RWMutex umožňuje získať zámok pre čítanie viacerými gorutinami, ale pre zápis umožňuje získať zámok jednej goroutine. Počas zápisu nie je umožnené získať zámok pre čítanie. Pri zápise čaká kým sú uvoľnené všetky zámky. Nasledujúci príklad demonštruje použitie RWMutexu.

```

1      var wg sync.WaitGroup
2      var rw sync.RWMutex
3
4      observer := func(i int) {
5          defer wg.Done()
6          rw.RLock()
7          fmt.Println("Reading started", i)
8          time.Sleep(400 * time.Millisecond)
9          fmt.Println("Reading ends", i)
10         rw.RUnlock()
11     }
12
13     writer := func(i int) {
14         defer wg.Done()
15         defer rw.Unlock()
16         rw.Lock()
17         fmt.Println("Writing started", i)
18         time.Sleep(400 * time.Millisecond)
19         fmt.Println("Writing ended", i)
20     }
21
22     for i := 0; i < 100; i++ {
23         wg.Add(1)
24         if i%25 == 0 {
25             go writer(i)
26         } else {
27             go observer(i)
28         }
29     }
30     wg.Wait()
31     fmt.Println("End")

```

Ukázkový kód 4: Použitie RWMutex

V ukázkovom kóde 4 na riadku dva si definujeme RWMutex. Na riadku štyri si definujeme funkciu čitateľa, ktorá sa pokúsi získať zámok pre čítanie a vypíše, že začala, počká, vypíše, že skončila a po skončení uvoľní zámok pre čítanie. Na riadku trinásť si definujeme funkciu zapisovateľa, ktorá sa pokúsi získať zámok, vypíše, že začala, počká, vypíše, že skončila a po skončení uvoľní zámok. Na riadku dvadsaťdva je cyklus, v ktorom voláme funkciu zapisovateľa ak i modulo 25 je nula, inak voláme funkciu čitateľa. Následne počkáme na dokončenie všetkých goroutín a vypíšeme, že sme skončili. Výsledok tohto kódu bude približne.

```
" Writting started 0"  
" Writting ended 0"  
" Reading started 1"  
" Reading started 2"  
" Reading started 5"  
...  
" Reading ended 98"  
" Reading ended 97"  
" Reading ended 99"  
" Writting started 50"  
" Writting ended 50"  
" Writting started 75"  
" Writting ended 75"
```

Z tohto výsledku vidíme, že RWMutex neuprednostňuje zapisovateľov. Vynútenie zámku pre zapisovateľa je možné pomocou ďalších synchronizačných nástrojov, ako napríklad pomocou komunikačných kanálov, kedy zapisovateľ pošle správu a hlavný proces nespustí ďalší proces čítania.

4 Chat server

Navrhnutý prototyp servera je založený na architektúre klient–server, v ktorej server zabezpečuje výmenu správ medzi pripojenými klientmi. Komunikácia medzi klientom a serverom je realizovaná prostredníctvom protokolu *Transmission Control Protocol* (TCP), ktorý zabezpečuje spoľahlivý prenos dát, zachovanie ich poradia a detekciu chýb pri prenose. Server je implementovaný v programovacom jazyku Go a využíva jeho vstavanú podporu súbežného programovania prostredníctvom goroutín. Hlavná úloha servera spočíva v prijímaní nových klientskych spojení, spracovaní požiadaviek klientov a správe komunikačných miestností.

4.1 Klientske spojenie

```
1      ln, err := net.Listen("tcp", ":8080")
2      for {
3          conn, err := ln.Accept()
4          if err != nil {
5              continue
6          }
7          go clientHandle.HandleConnection(conn)
8      }
```

Zdrojový kód 5: Klientské spojenie

V kóde 5 pri spustení server vytvorí TCP poslucháč `ln` na riadku jedna. Na riadku tri v nekonečnom cykle server čaká na nové pripojenia. Ak je spojenie úspešne vytvorené, server preň spustí novú goroutinu na riadku sedem. Tým sa zabezpečí, že každý klient je obsluhovaný súbežne bez blokovania prijímania ďalších klientskych spojení.

Každý klient je obsluhovaný v samostatnej goroutine, ktorá vykonáva funkciu *HandleConnection*. Z pohľadu architektúry toto riešenie predstavuje nasledujúci model:

- jeden hlavný serverový proces,
- jedna goroutina pre každého klienta.

Riešenie je efektívne, pretože goroutiny majú v porovnaní s klasickými systémovými vláknami menšie režijné náklady a umožňujú súbežne obsluhovať veľký počet klientov.

Po pripojení server vyžiada od klienta používateľské meno. Následne klienta pripojí do komunikačnej miestnosti a v nekonečnom cykle spracúva jeho požiadavky:

- `/fetch` – získanie nových správ z aktuálnej komunikačnej miestnosti,
- `/fetchAll` – získanie správ zo všetkých navštívených komunikačných miestností,
- `/broadcast` – odoslanie správy do všetkých navštívených komunikačných miestností,
- `exit` – opustenie aktuálnej komunikačnej miestnosti,
- `logout` – odhlásenie zo servera a ukončenie cyklu spracúvania požiadaviek,
- `default` – odoslanie správy do aktuálnej komunikačnej miestnosti.

4.2 Algoritmus posielania správ

Pri posielaní správ je potrebné zabezpečiť, aby sa jednotlivé správy správne uložili a aby pri súbežnom prístupe viacerých klientov nevznikol problém súbehu. Na riešenie tohto problému bola použitá komunikácia medzi goroutinami prostredníctvom komunikačného kanála. Využíva sa pritom jeho blokujúce správanie a schopnosť bezpečne prenášať údaje medzi goroutinami.

Na reprezentáciu správy bola použitá štruktúra *Message*, v ktorej sa uchováva meno odosielateľa a obsah správy. Pre každú komunikačnú miestnosť bola vytvorená štruktúra *ChatLog*, ktorá obsahuje zoznam odoslaných správ typu *Message*. Súčasťou tejto štruktúry je aj komunikačný kanál, prostredníctvom ktorého sa nové správy odovzdávajú goroutine zodpovednej za ich ukladanie.

```

1         for msg := range cl.writeCh {
2             cl.Message = append(cl.Message, msg)
3         }

```

Zdrojový kód 6: Prijatie správy

V kóde 6 môžeme vidieť cyklus, v ktorom čakáme na prijatie správy z komunikačného kanála. Po prijatí správy ju pridá do zoznamu správ príslušnej komunikačnej miestnosti. Ak chce klient odoslať správu, vloží ju do komunikačného kanála:

```

1         cl.writeCh <- message.Message{
2             User: user,
3             Message: text,
4         }

```

Zdrojový kód 7: Poslanie správy

V kóde 7 vidíme poslanie správy ako štruktúru *Message*. Tá sa spracuje ako je ukázané v kóde 6.

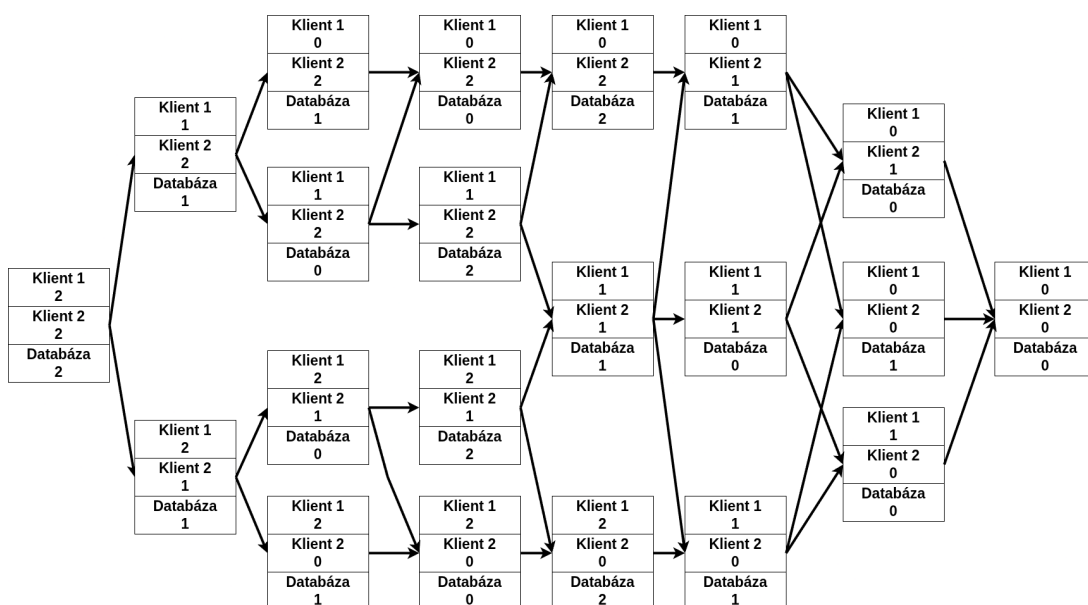
Ak sa o odoslanie správy pokúsi viacero klientov súčasne, jednotlivé operácie odoslania sú spracované postupne. Pri nebufferovanom kanáli zostane odosielajúca goroutina zablokováaná dovtedy, kým prijímajúca goroutina nebude pripravená správu prevziať. Tým sa zabezpečí, že zoznam správ upravuje iba jedna goroutina a nedochádza k súbežnému zápisu do tej istej dátovej štruktúry. Uvedený postup možno zapísať nasledujúcim pseudokódom:

go handler() go channel()	
ch <- message return	for { message := <-ch save(message) }

Obr. 1: Pseudokód posielania správ

Pre ukladanie dát je vytvorená samostatná goroutina, ktorá prijíma správy z kanála a zapisuje ich do úložiska správ. Ďalšie goroutiny zabezpečujú obsluhu jednotlivých klientov. Aj pri súbežnom odosielaní správ viacerými klientmi nevzniká súbežný zápis do zoznamu, pretože všetky správy spracúva jedna prijímajúca goroutina.

Využíva sa pritom blokujúce správanie komunikačného kanála. Ak sa viaceré goroutiny pokúsia odoslať správu, odosielajúca goroutina čaká, kým prijímajúca goroutina nebude pripravená údaje prevziať. Tento postup je znázornený na príklade dvoch klientskych procesov a jedného procesu zabezpečujúceho ukladanie správ.



Obr. 2: Synchronizácia klientov pomocou komunikačného kanála

Na obrázku 2 sú znázornené procesy dvoch klientov posielajúcich správu ako Klient 1 a Klient 2 a jeden proces obsluhujúci zápis klientských správ ako Databáza. Proces klientov pozostáva z krokov poslanie správy cez komunikačný kanál, pokračovanie a ukončenie, ktoré su označené ciframi dva, jedna a nula. Proces obsluhujúci databázu vykonáva kroky prijatie správy cez komunikačný kanál, uloženie správy a opakovanie cyklu, označené ciframi dva, jedna a nula. Z obrázka vyplýva, že klientske procesy zostávajú blokované dovtedy, kým proces ukladania dát nie je pripravený údaje prijať. Prenos údajov prebieha synchronne v zodpovedajúcich stavoch procesu databázy a klienta. Mimo tejto komunikácie sa jednotlivé procesy vykonávajú nezávisle. Takýmto spôsobom je zabezpečené korektné poradie prijímania a ukladania správ.

4.3 Algoritmus získania správ

Pri získavaní správ je potrebné zabezpečiť, aby boli všetky správy získané v neporušenom stave a následne odoslané klientovi. Na synchronizáciu boli použité mechanizmy *Mutex* a *WaitGroup*.

```

1      var chatLog chatlog.ChatLog
2
3      for _, room := range rooms {
4          wg.Add(1)
5
6          go func(room string, chatLog *chatlog.ChatLog) {
7              defer wg.Done()
8
9              mutex.Lock()
10             getMessages()
11             mutex.Unlock()
12
13             mapMessages()
14         }(room, &chatLog)
15     }
16
17     wg.Wait()

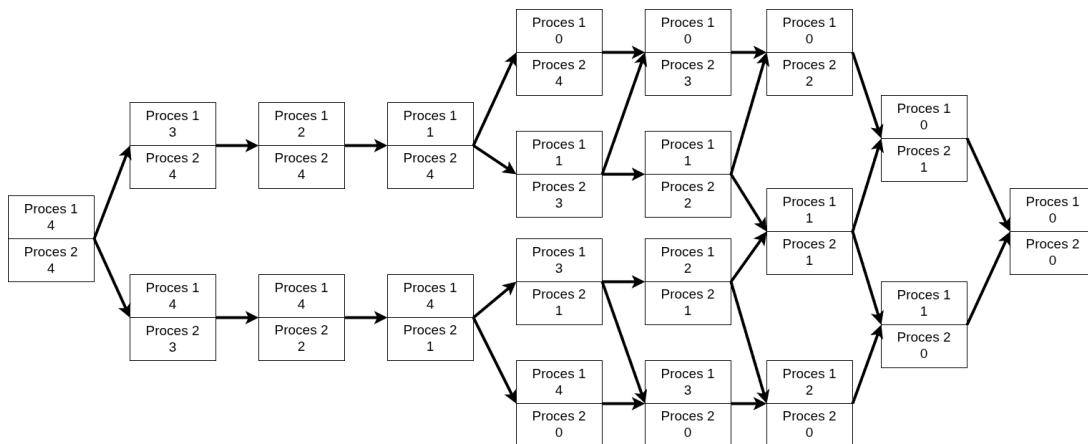
```

Zdrojový kód 8: Získanie správ

V kóde 6 sa pre každú navštívenú komunikačnú miestnosť spustí samostatná goroutina na riadku šesť. Na riadku štyri počítadlo mechanizmu *WaitGroup* zvýšime o hodnotu jeden. Každá goroutina následne získa zámok na riadku deväť. Na riadku desať načíta správy, uvoľní zámok na riadku jedenásť. Po uvoľnení zámku načítané správy namapuje do odpovede určenej klientovi. Hlavná goroutina čaká prostredníctvom operácie *wg.Wait()*, kým všetky spustené goroutiny dokončia svoju činnosť.

Správy sa získavajú prostredníctvom komunikačného kanála, ktorý je definovaný ako odkaz na príslušnú štruktúru. *Mutex* je preto potrebný na ochranu

kritickej sekcii, aby počas získavania správ nedošlo k zmene zdieľaného odkazu alebo k nekorektnému súbežnému prístupu. Bez tejto synchronizácie by mohlo dôjsť k vynechaniu alebo neželanému duplikovaniu niektorých správ.



Obr. 3: Ochrana kritickej sekcii pomocou mutexu

Na obrázku 3 definujeme `Proces 1` a `Proces 2`, ktoré vykonávajú kroky získanie zámku, získanie správ, uvoľnenie zámku a ukončenie, označené ciframi štyri, tri, dva a jedna. Jednotlivé procesy nemôžu súčasne vstúpiť do kritickej sekcii, ktorú predstavuje získanie správ. Pred vstupom do nej získajú zámok a po vykonaní potrebných operácií ho uvoľnia. Mimo kritickej sekcii môžu procesy pokračovať nezávisle. Použitím mutexu sa zabezpečí, že sa v kritickej sekcii v danom okamihu nachádza iba jeden proces, čím sa predchádza nekorektnému súbežnému prístupu k zdieľaným údajom.

4.4 Algoritmus príkazu `select`

Príkaz `select` bol použitý na strane klienta pri periodickom získavaní správ z komunikačnej miestnosti.

```

1      select {
2      case <-ch:
3          return
4      default:
5          getMessages()
6          time.Sleep(1 * time.Second)
7      }
```

Zdrojový kód 9: Implementácia príkazu `select`

Pri pripojení klienta do komunikačnej miestnosti sa vytvorí komunikačný kanál, ktorý sa používa na signalizáciu ukončenia periodického získavania správ.

V kóde 9 príkaz `select` má jednu vetvu `case` a druhú vetvu `default`. Prvá podmienka na riadku dva kontroluje zatvorenie komunikačného kanála. Po opustení komunikačnej miestnosti sa kanál uzavrie. Uzavretím kanála sa neposiela samostatná správa, ale operácia prijímania z uzavretého kanála sa môže okamžite vykonať. Príkaz `select` preto vyberie vetvu `case <-ch:`, ktorá ukončí funkciu zabezpečujúcu periodické získavanie nových správ. Vetva `default`, v ktorej klient požiada server o nové správy a následne počká jednu sekundu sa vykoná vždy pokiaľ nie je prvá podmienka splnená.

5 Ďalšie možnosti použitia

Navrhnutý prototyp chat servera môže slúžiť ako základ komplexnejšej komunikačnej platformy. Riešenie možno rozšíriť o trvalé databázové úložisko správ, autentifikáciu a autorizáciu používateľov, správu používateľských účtov, šifrovanú komunikáciu a podrobnejšie riadenie prístupových práv v komunikačných miestnostiach. Ďalším možným smerom je podpora prenosu súborov, upozornení, histórie správ alebo komunikácie medzi viacerými servermi. Vďaka oddeleniu obsluhy klientov do samostatných goroutín a využitiu kanálov na výmenu údajov možno prototyp postupne rozširovať bez zásadnej zmeny jeho základného modelu súbežnosti.

Mechanizmy súbežného a paralelného programovania v jazyku Go možno využiť aj mimo oblasti komunikačných serverov. Uplatnenie nachádzajú pri spracovaní veľkého množstva nezávislých požiadaviek, tvorbe webových služieb, distribuovaných systémov, sieťových nástrojov, dátových potrubí a aplikácií pracujúcich s prúdmi údajov. Goroutiny umožňujú rozdeliť úlohu na menšie súbežne vykonávané časti, zatiaľ čo komunikačné kanály poskytujú prostriedok na ich koordináciu a bezpečnú výmenu údajov. Paralelné spracovanie možno využiť napríklad pri spracovaní súborov, výpočtoch nad veľkými množinami údajov, obsluhu sieťových spojení alebo vykonávaní viacerých nezávislých operácií súčasne.

6 Záver

Cieľom bakalárskej práce bolo priblížiť princípy paralelného a súbežného programovania v jazyku Go so zameraním na model *Communicating Sequential Processes* a opísať mechanizmy, ktoré jazyk Go poskytuje na tvorbu súbežných aplikácií. Teoretická časť práce predstavila základné koncepty modelu CSP, goroutiny, komunikačné kanály a synchronizačné mechanizmy. Ich praktické využitie bolo demonštrované návrhom a implementáciou prototypu chat servera schopného súbežne obsluhovať viacerých klientov.

Implementovaný prototyp ukázal, že model súbežnosti jazyka Go umožňuje prirodzene rozdeliť sieťovú aplikáciu na samostatné goroutiny zodpovedné za obsluhu klientov, spracovanie správ a synchronizáciu zdieľaných údajov. Komunikačné kanály umožnili riadiť prenos správ medzi jednotlivými časťami systému bez potreby priameho zdieľania a súbežnej modifikácie dát. Mutex a mechanizmus WaitGroup boli použité v prípadoch, v ktorých bolo potrebné chrániť kritickú sekciu alebo čakať na dokončenie viacerých súbežných operácií.

Výhodou jazyka Go je, že podpora súbežnosti nie je iba doplnkom poskytovaným externými knižnicami, ale je priamo súčasťou návrhu jazyka a jeho runtime prostredia. Goroutiny majú jednoduchý spôsob vytvárania a umožňujú obsluhovať jednotlivé úlohy oddelene, zatiaľ čo kanály poskytujú zrozumiteľný model ich komunikácie a synchronizácie. V porovnaní s riešeniami založenými výlučne na priamom riadení systémových vlákien a ručnej ochrane všetkých zdieľaných údajov môže tento prístup viesť k prehľadnejšiemu kódu a jednoduchšiemu odôvodneniu jeho správnosti.

Praktická časť túto výhodu demonštrovala na obsluhu klientskych spojení, bezpečnom ukladaní správ a súbežnom získavaní údajov z viacerých komunikačných miestností. Rozdelenie zodpovedností medzi samostatné goroutiny spolu s použitím kanálov a synchronizačných mechanizmov umožnilo vytvoriť riešenie, v ktorom možno presne určiť miesta komunikácie, blokovania a prístupu k zdieľaným údajom. Tým sa znižuje riziko vzniku problémov súbehu a zároveň sa zvyšuje zrozumiteľnosť implementovaného algoritmu.

Výsledky práce preto poukazujú na to, že jazyk Go je vhodným nástrojom na vývoj súbežných sieťových aplikácií. Jeho mechanizmy umožňujú kombinovať efektívnu obsluhu väčšieho počtu úloh s relatívne jednoduchým programovým modelom. Práve spojenie goroutín, komunikačných kanálov a mechanizmov runtime prostredia predstavuje významnú výhodu Go pri implementácii systémov, v ktorých sú komunikácia, synchronizácia a súbežné spracovanie základnými požiadavkami.

7 Conclusion

The aim of this bachelor's thesis was to introduce the principles of parallel and concurrent programming in the Go programming language, with a focus on the *Communicating Sequential Processes* model, and to describe the mechanisms that Go provides for the development of concurrent applications. The theoretical part of the thesis presented the fundamental concepts of the CSP model, goroutines, communication channels, and synchronization mechanisms. Their practical use was demonstrated through the design and implementation of a prototype chat server capable of serving multiple clients concurrently.

The implemented prototype showed that Go's concurrency model makes it possible to divide a network application naturally into separate goroutines responsible for client handling, message processing, and the synchronization of shared data. Communication channels made it possible to control the transfer of messages between individual parts of the system without the need for direct sharing and concurrent modification of data. A mutex and the WaitGroup mechanism were used in situations where it was necessary to protect a critical section or wait for the completion of multiple concurrent operations.

One of the advantages of Go is that support for concurrency is not merely an extension provided by external libraries, but is directly integrated into the design of the language and its runtime environment. Goroutines can be created easily and allow individual tasks to be handled separately, while channels provide a clear model for communication and synchronization between them. Compared with solutions based exclusively on the direct management of operating-system threads and the manual protection of all shared data, this approach can lead to clearer code and make it easier to reason about its correctness.

The practical part demonstrated this advantage through the handling of client connections, the safe storage of messages, and the concurrent retrieval of data from multiple communication rooms. The division of responsibilities among separate goroutines, together with the use of channels and synchronization mechanisms, made it possible to create a solution in which the points of communication, blocking, and access to shared data could be identified precisely. This reduces the risk of concurrency-related problems while also improving the clarity of the implemented algorithm.

The results of the thesis therefore indicate that Go is a suitable tool for developing concurrent network applications. Its mechanisms make it possible to combine the efficient handling of a large number of tasks with a relatively simple programming model. The combination of goroutines, communication channels, and runtime mechanisms represents a significant advantage of Go when implementing systems in which communication, synchronization, and concurrent processing are fundamental requirements.

A Obsah elektronických dát

Elektronické dáta tejto bakalárskej práce sú rozdelené do nasledujúcej štruktúry adresárov a súborov:

README.md

Textový súbor obsahujúci informácie o požiadavkách na prostredie, postupe zostavenia a spustenia aplikácie, štruktúre projektu a popise podporovaných príkazov klienta.

source.zip

ZIP archív obsahujúci kompletne zdrojové kódy implementovanej klient-server aplikácie vrátane definície Go modulu a všetkých súborov potrebných na zostavenie projektu.

/text

Adresár obsahujúci text bakalárskej práce vo formáte PDF a ZIP archív so zdrojovými súbormi práce, ktoré sú potrebné na vytvorenie výsledného PDF dokumentu.

Literatúra

- [1] Hoare, C. A. R. *Communicating Sequential Processes*. Englewood Cliffs, NJ: Prentice-Hall International, 1978.
- [2] Hoare, C. A. R. Communicating Sequential Processes. In Graham, S. L.; Rivest, R. L. (ed.). *Programming Techniques*. New York: Academic Press, 1978.
- [3] Baugh, John P. *Go Programming*. 2010. ISBN 9781453636671.
- [4] Beckingsale, David. *Towards Scalable Adaptive Mesh Refinement on Future Parallel Architectures*. 2015.
- [5] Cox-Buday, Katherine. *Concurrency in Go*. First. Sebastopol, CA: O'Reilly Media, 2017. 224 str. ISBN 978-1-491-94119-5.
- [6] Go Authors. *Go Runtime Source Code: runtime/chan.go*. 2026. Part of the Go programming language runtime implementation. Accessed 2026-05-02. <https://go.dev/src/runtime/chan.go> (<https://go.dev/src/runtime/chan.go>).
- [7] Go Authors. *Go Runtime Source Code: runtime/select.go*. 2026. Part of the Go programming language runtime implementation. Accessed 2026-05-02. <https://go.dev/src/runtime/select.go> (<https://go.dev/src/runtime/select.go>).
- [8] Go Authors. *Go Runtime Source Code: time*. 2026. Part of the Go programming language runtime implementation. Accessed 2026-05-02. <https://pkg.go.dev/time> (<https://pkg.go.dev/time>).
- [9] Go Authors. *Go Runtime Source Code: sync*. 2026. Part of the Go programming language runtime implementation. Accessed 2026-05-02. <https://pkg.go.dev/sync> (<https://pkg.go.dev/sync>).